

Summary

I spent 3 days overall working on this. I dedicated a majority of the time that I could allocate to this, but due to unfortunate circumstances, I lost a majority of my work on the day it was due and was allotted an extension. I underestimated how long it would take me to rebuild the program after losing the data, and that led to now.

Overall I think the hardest part for me was coming up with the unit tests and mapping it all out in my head, it took a long time for me to plan and build, and having to redo from scratch, where I worked on this nonstop was not good mentally.

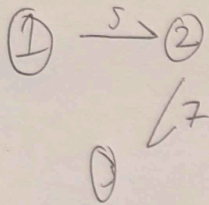
Either way, GraphM's adjacency matrix was coded in a way for linear scans being $O(V^2)$ with $O(V^3)$ for finding the shortest path for all pairs. I made recursive constructors for displayPath and getPath.

I also implemented GraphL, I made test cases for all things I think could fail under driver.cpp. Finally I made race and tracking under gen_graphs that I output to race, which runs up to a graph of 3200 vertices.

Manual Trace vs Program Output

graph 1

src 1:



coffee
from 1
to 2
3

dist path
5 [1,2]
12 [1,2,3]

graph 2

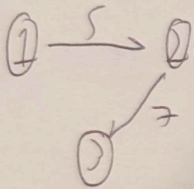
bfs

1, 2, 3

bfs

1, 2, 3

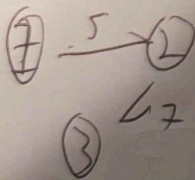
src 2:



library 7 repr

from 2 to 1 dist inf path --
2 2 7 [1,2]

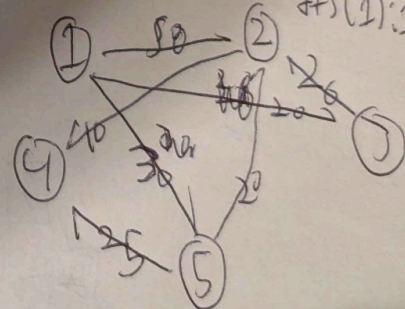
src 3:



from 3 to 1 dist inf path --
3 2 inf --

graph 2

bfs(1): 1, 2, 3, 4, 5
bfs(2): 1, 2, 4, 3, 5



```

=== GraphM · graph 2 (3 nodes) ===
Description      From To Distance Path
Coffee cart
    1    2        5  1 2
    1    3       12  1 2 3
Library steps
    2    1       ---
    2    3        7  2 3
Fountain lawn
    3    1       ---
    3    2       ---

```

```

=== GraphL · graph 1 (5 nodes) ===
Graph:
Node 1      Aurora and 85th
    edge 1 5
    edge 1 3
    edge 1 2
Node 2      Green Lake Starbucks
    edge 2 4
Node 3      Woodland Park Zoo
    edge 3 2
Node 4      Troll under Aurora bridge
Node 5      PCC
    edge 5 4
    edge 5 2
edgeList(1): 5 3 2
DFS from 1:  1 2 4 3 5
BFS from 1:  1 5 3 2 4

```

```

=== GraphL · graph 2 (3 nodes) ===
Graph:
Node 1      Coffee cart
    edge 1 2
Node 2      Library steps
    edge 2 3
Node 3      Fountain lawn
edgeList(1): 2
DFS from 1:  1 2 3
BFS from 1:  1 2 3

```

Race

I think something was wrong with my program. I ran it with `g++ -std=c++17 -g` and I couldn't get the numbers to overlap as much as I tried, GraphM kept accelerating faster, and I feel like it could have been something with my docker container.

Either way, in theory I guess Dijkstra would approach $O(V^3 \log(V))$ thus being way bigger than $O(V^3)$, and it should have been overtaken, but I feel like I've run out of ideas on what went wrong here. I feel like if I had a lot more time, and a faster computer, I would be able to see the extremes, but I don't have the tech for it.

Here are the numbers

Type	V	GraphM (ms)	09 Dijkstra (ms)
Sparse	200	41.9115 ms	6.55902 ms
Dense	200	73.1877 ms	54.7727 ms
Sparse	400	264.958 ms	27.2711 ms
Dense	400	568.264 ms	307.041 ms
Sparse	800	1898.78 ms	131.444 ms
Dense	800	4368.64 ms	1859.16 ms
Sparse	1600	15818.6 ms	544.191 ms
Dense	1600	33875.5 ms	12240.1 ms
Sparse	3200	167018 ms	2295.35 ms
Dense	3200	597736 ms	81545.7 ms

AI disclosure

AI was used when I googled depth first search, when attempting to recall past information, and attempting to pull up something on the geeks for geeks website. At worst, AI was used when having it explain concepts, or clarify questions to deepen my understanding. AI was NOT used to write code, pseudocode, or anything else, what I wrote was 100% me.

Task log

- Sunday- 8/2/26
 - Complete graphM ~6 hours
- Monday - 8/3/26
 - Complete graphL ~4 hours
 - built unit tests, fixing bugs ~10 hours

- Tuesday - 8/4/26
 - Worked on unit tests ~2 hours
 - Worked on race in gen_graphs ~ 30 mins
 - Dropped laptop resetting my work back to halfway into graphM, due to BSOD
- Wednesday
 - Got laptop working ~2 hours, couldn't recover files.
 - rebuilt graphM ~ 2 hours
 - rebuilt graphL ~ 1 hour
 - rebuilt unit tests ~ 7 hours //took me the longest really.
 - race + tests ~ 2 hours
 - report ~ 2 hours //exhaustion is getting to me.